Behind the Scenes of E-Learning: Developing Backend for Education Apps

This is a short introduction about e-learning platforms' significance in the present-day education system and how backend development is a dominant factor in the success of these applications. This is our own company education app.

We trust a good backend infrastructure design is reliable, scalable and secure for students, teachers and admins.

Technologies Used (Backend)

Node.js: This technology is used for designing the server-side applications.

Express.js: An API routes handling framework.

MongoDB: A database used for its NoSQL flexible schema (study materials, user profiles).

Mongoose: MongoDB object data modeling library that implements the schema and its relations.

JWT (JSON Web Token): A mechanism for safe user authentication.

APP Overview

Sign Up Screen with User Registration/Sign In/Sign In with OTP:-

This is how you could design the UI for the Sign Up/Sign In page with model and schemas of behind this page.

Sign Up Page:-

Model & Schemas Behind Signup Screen

Model:-

```
337  export const createChild = (req, res) => {
338    const {
339      parentid,
340      stage,
341      age,
342      phone,
343      email,
344      alterphone,
345      password,
346      schoolname,
347      address,
348      language,
349      gender,
350      image,
351      imagename,
352      boardname,
353      fathername,
354      mothername,
355      name,
356      isPremium,
357      subscriptionStartDate,
358      subscriptionEndDate,
359      lname,
360      fname,
361      boardid,
362      stageid,
363      classname,
364      statename,
365      stateid
366    } = req.body;
367    // const childid = "WELL" + name.substr(fname.length - 1) +  Date.now()
368    edChild.findOne({ phone: phone }, (err, user) => {
369      if (user) {
370        return res.send({ message: "User already registered", status: false });
371      } else {
372        const childid = "WELL" + Date.now();
373        const user = new edChild({
```

Controller:-

```
337  export const createChild = (req, res) => {
338    const {
339      parentid,
340      stage,
341      age,
342      phone,
343      email,
344      alterphone,
345      password,
346      schoolname,
347      address,
348      language,
349      gender,
350      image,
351      imagename,
352      boardname,
353      fathername,
354      mothername,
355      name,
356      isPremium,
357      subscriptionStartDate,
358      subscriptionEndDate,
359      lname,
360      fname,
361      boardid,
362      stageid,
363      classname,
364      statename,
365      stateid
366    } = req.body;
367    // const childid = "WELL" + name.substr(fname.length - 1) +  Date.now()
368    edChild.findOne({ phone: phone }, (err, user) => {
369      if (user) {
370        return res.send({ message: "User already registered", status: false });
371      } else {
372        const childid = "WELL" + Date.now();
```

```
337  export const createChild = (req, res) => {
368    edChild.findOne({ phone: phone }, (err, user) => {
369      if (user) {
370        return res.send({ message: "User already registered", status: false });
371      } else {
372        const childid = "WELL" + Date.now();
373        const user = new edChild({
374          childid,
375          parentid,
376          stage,
377          phone,
378          email,
379          alterphone,
380          password,
381          schoolname,
382          address,
383          language,
384          gender,
385          image,
386          imagename,
387          name,
388          age,
389          boardname,
390          fathername,
391          mothername,
392          isPremium,
393          subscriptionStartDate,
394          subscriptionEndDate,
395          lname,
396          fname,
397          boardid,
398          stageid,
399          classname,
400          statename,
401          stateid
402        });
403        user.save((err, suc) => {
```

When Child Signup using Password:-

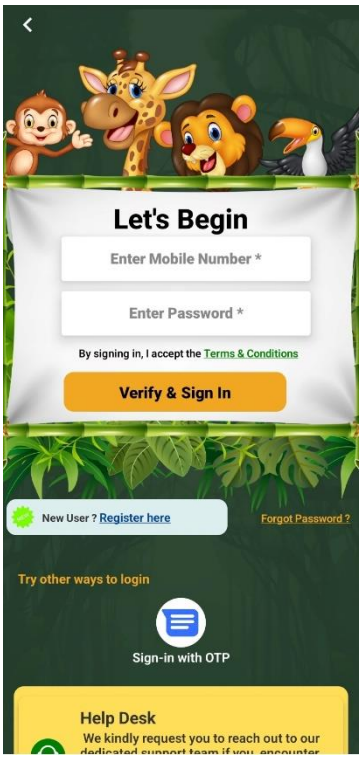
Verification: Verify if the given inputs in the required fields are correct.

Password Security: Passwords should satisfy certain requested (for example minimum length and/or complexity).

Store Data: After the validation, the password will be secured using a hashing algorithm such as bcrypt before being persisted in the database.

Account Creation: After the password is hashed, and data verified, the child account is created and the information saved in the database.

Token Generation: One can also create a JWT (JSON Web Token) that allows one to login immediately after they finish signing up.



Controller for Login using Password:-

This controller show aggregation and lookup required for while user login through using Password.

```
export const childlogiusingpassword = (req, res) => {
  const { phone, password } = req.body;
  edChild.findOne({ phone: phone }, (err, user) => {
    if (user) {
      if (password === user.password) {
        edChild
          .aggregate([
            {
              $match: {
                phone: phone,
              },
            },
            {
              $lookup: {
                from: "edmasterscholarschemas",
                let: { stageid: "$stageid", boardid: "$boardid" },
                pipeline: [
                  {
                    $match: {
                      $expr: {
                        $and: [
                          { $eq: ["$stageid", "$$stageid"] },
                          { $eq: ["$boardid", "$$boardid"] },
                        ],
                      },
                    },
                  },
                  {
                    $sort: { scholarshipname: 1 },
                  },
                ],
                as: "scholarship",
              },
            },
          ])
          .then((succ, error) => {
            if (succ) {
```

```
              const token =  generateToken(succ[0]._id)
              edUserDeviceDetails.findOne({childid:succ[0].childid},(err,prod)=>{
                if(prod){
                  const updateData = edUserDeviceDetails.findOneAndUpdate(
                    {childid:succ[0].childid },
                    {
                      devicetoken: token || findData.devicetoken,
                      updatedon: Date.now()
                    },
                    (err, sucUpdate) => {
                    }
                  );
                }else{
                    const UserDeviceDetails = new edUserDeviceDetails({
                        childid:succ[0].childid,  devicetoken:token,
                    });
                    UserDeviceDetails.save((err)=>{
                        if (err) {
                        }else{
                        }
                    })
                }
            })
              return res.json({
                user: succ,
                status: true,
                message: "verified Successfully",
                authtoken: token,
              });
            } else {
              return res.send({
                message: "verified Successfully",
                newUser: true,
                status: true,
```

```
          } else {
            return res.send({
              message: "Password didn't match",
              newUser: false,
              status: false,
            });
          }
        } else {
          return res.send({
            message: "User not registered",
            newUser: true,
            status: true,
          });
        }
      });
    };
```

How To Login with OTP:-

Most applications incorporate the functionality of One-Time Passwords (OTPs) in order to authenticate users alike at the point of logging in, signing up or conducting any sensitive actions such as changing one's password. Most of all in education apps where a user needs to firmly log in.

The controller 'Get OTP' handles the aspects of creating, and sending an OTP, generated during the user registration of login to the registered user's mobile number or email. It also makes sure that the OTP is kept safely and checked under the provided time.

Controller for Get OTP:-

```
export const getOtp = async (req, res) => {
  const { phone, senderid, status, otp } = req.body;
  let patientPhone = phone;
  if (phone.length > 10) {
    if (phone.includes("+91")) {
      patientPhone = phone.slice(3);
    } else {
      patientPhone = phone.slice(2);
    }
  }
  Otp.findOne({ phone: patientPhone }, (err, suc) => {
    if (suc) {
      if (otp == suc.otp) {
        masterPatient.findOne({ phone: patientPhone }, (err, usuc) => {
          if (usuc) {
            const token =  generateToken(usuc[0]._id)

            edUserDeviceDetails.findOne({childid:usuc[0].childid},(err,prod)=>{

              if(prod){
                const updateData = edUserDeviceDetails.findOneAndUpdate(
                  {childid:usuc[0].childid },
                  {
                    devicetoken: token || findData.devicetoken,
                    updatedon: Date.now()
                  },
                  (err, sucUpdate) => {
                    // if (!sucUpdate) return res.json({ error: "Something went wrong", status: false  });
                    // else return res.json({ message: "User device details updated usucessfully", status: true  });
                  }
                );
              }else{
                  const UserDeviceDetails = new edUserDeviceDetails({
                      childid:usuc[0].childid,  devicetoken:token,
                  });
                  UserDeviceDetails.save((err)=>{
                      if (err) {
```

```
                return res.send({
                  message: "OTP verified Successfully",
                  user: usuc,
                  authtoken: token,
                  newUser: false,
                  status: true,
                });
              }
              return res.send({
                message: "OTP verified Successfully",
                newUser: true,
                status: true,
              });
            });
          } else {
            return res.send({ message: "OTP didn't match", status: false });
          }
        } else {
          return res.send({ message: "Phone number not found", status: false });
        }
      });
};
```

How to get Subject Progress Details:

Now we acquire the entire information of the progress and it is clear how many percentage the user finishes the subject.

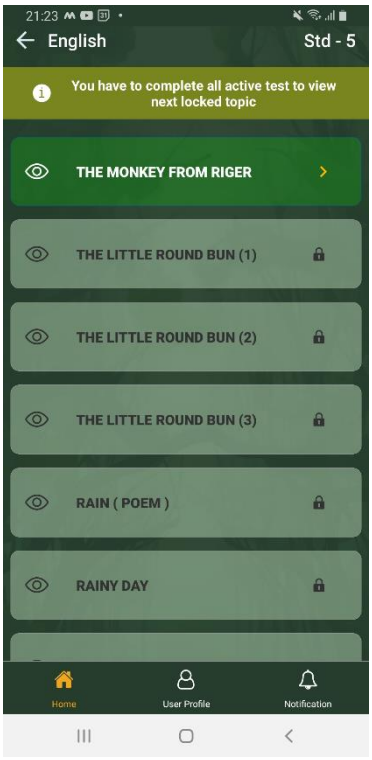
Controller for the Subject Progress Details:-

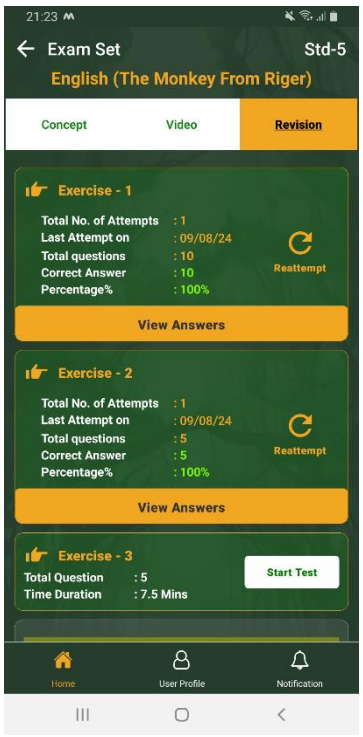
The role of this controller is to retrieve details on a child's progress using various parameters like stageid, boardid, scholarshipid and childid. It extracts subject progress on the number of total exercises available and the number of exercises completed and derives the completion percentage of the given subject.

```
export const getChildonProgressDetilsone = (req, res) => {
  const { stageid, boardid, scholarshipid, childid } = req.params;
  edSubject.aggregate([
    {
      $match: {
        stageid: stageid,
        boardid: boardid,
        scholarshipid: scholarshipid,
      },
    },
    {
      $lookup: {
        from: "edcontentmasterschemas",
        let: {
          stageid: "$stageid",
          subjectid: "$subjectid",
          boardid: "$boardid",
          scholarshipid: "$scholarshipid",
        },
        pipeline: [
          {
            $match: {
              $expr: {
                $and: [
                  { $eq: ["$stageid", "$$stageid"] },
                  { $eq: ["$subjectid", "$$subjectid"] },
                  { $eq: ["$boardid", "$$boardid"] },
                  { $eq: ["$scholarshipid", "$$scholarshipid"] },
                ],
              },
            },
          },
          {
            $group: {
              _id: null,
              totalExercises: { $sum: 1 },
              completedExercises: {
                $sum: { $cond: [{ $eq: ["$completed", true] }, 1, 0] },
              },
            },
          },
        ],
        as: "exerciseCompletion",
      },
    },
    {
      $project: {
        totalExercises: { $arrayElemAt: ["$exerciseCompletion.totalExercises", 0] },
        completedExercises: { $arrayElemAt: ["$exerciseCompletion.completedExercises", 0] },
      },
    },
    {
      $addFields: {
        percentageCompletion: {
          $multiply: [
            { $divide: ["$completedExercises", "$totalExercises"] },
            100,
          ],
        },
      },
    },
    {
      $group: {
        _id: "$_id",
        subject: { $first: "$subject" },
        subjectid: { $first: "$subjectid" },
        slsubject: { $first: "$slsubject" },
        subjectIamge: { $first: "$subjectImage" },
        percentageCompletion: { $first: "$percentageCompletion" },
      },
    },
    {
      $project: {
        _id: 1,
        subject: 1,
        subjectid: 1,
        slsubject: 1,
        subjectImage: 1,
        percentageCompletion: 1,
      },
    },
  ])
    .sort({ slsubject: 1 })
    .collation({
      locale: "en_US",
      numericOrdering: true,
    })
    .then((succ, error) => {
      if (succ) {
        return res.json({ data: succ, status: true });
      } else {
        return res.status(404).json({ error: "No Content.", status: false });
      }
    });
};
```

How to added Content/Topic/Questions:-

While developing any education oriented mobile application focusing on content, topics, questions (especially for competitive exams like Navodaya entrance exam) one needs to create a well-organized backend api to support the addition of new educational contents. This will enable the application administrators, educators or some other automated systems to insert additional content such as subjects, topics, at a later time for the students.



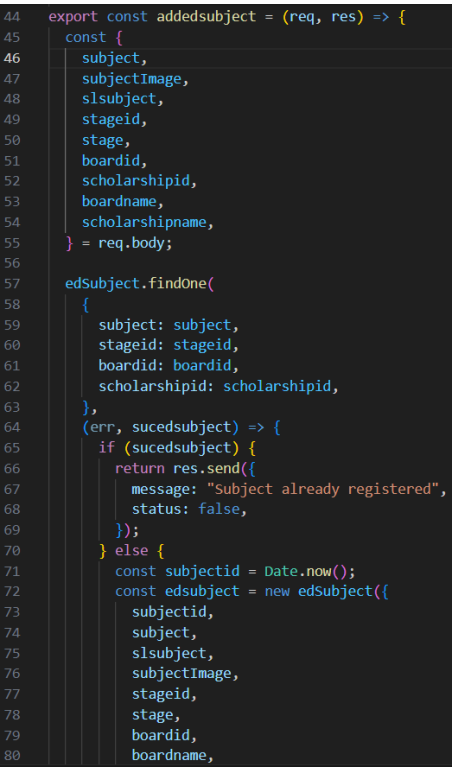


Controller of How to added Content/Topic/Questions:-

This controller function is responsible for adding a new subject into the Subject collection in MongoDB.

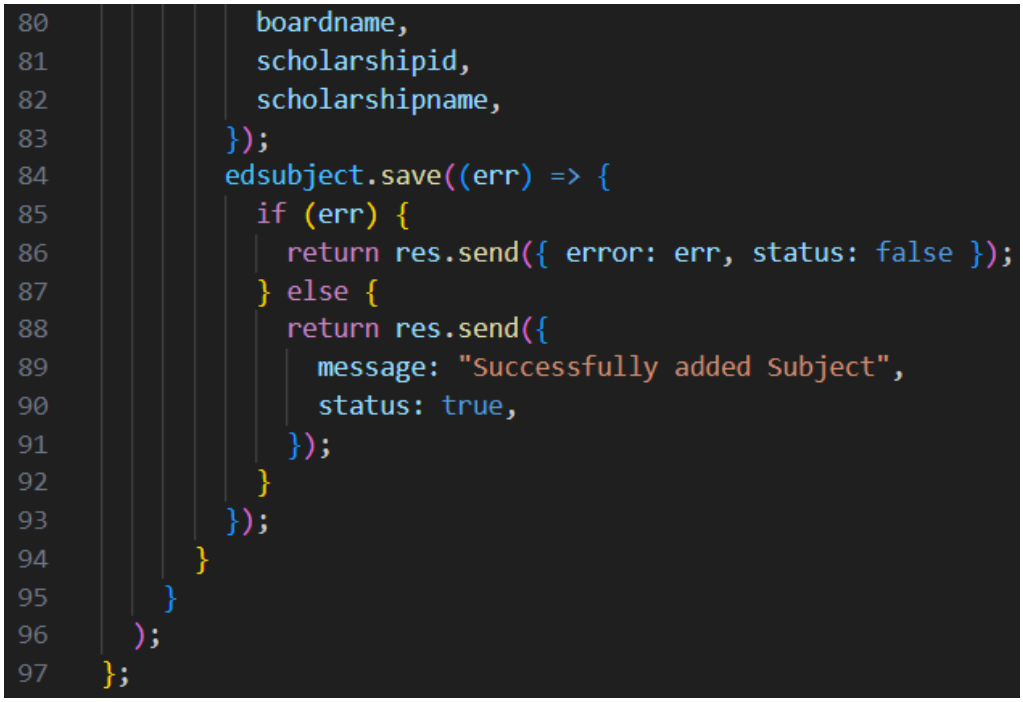
The purpose of this function is to register a new subject into the system. Before registering, it checks whether the subject already exists based on the combination of subject, stageid, boardid, and scholarshipid. If the subject is already registered, the user is notified with an error message; otherwise, the subject is added to the database.

Controller for Added Subject:-



```
export const addedsubject = (req, res) => {
  const {
    subject,
    subjectImage,
    slsubject,
    stageid,
    stage,
    boardid,
    scholarshipid,
    boardname,
    scholarshipname,
  } = req.body;

  edSubject.findOne(
    {
      subject: subject,
      stageid: stageid,
      boardid: boardid,
      scholarshipid: scholarshipid,
    },
    (err, sucedsubject) => {
      if (sucedsubject) {
        return res.send({
          message: "Subject already registered",
          status: false,
        });
      } else {
        const subjectid = Date.now();
        const edsubject = new edSubject({
          subjectid,
          subject,
          slsubject,
          subjectImage,
          stageid,
          stage,
          boardid,
          boardname,
```



```
          boardname,
          scholarshipid,
          scholarshipname,
        });
        edsubject.save((err) => {
          if (err) {
            return res.send({ error: err, status: false });
          } else {
            return res.send({
              message: "Successfully added Subject",
              status: true,
            });
          }
        });
      }
    }
  );
};
```

Controller for Added Content:-

The createMasteredcontent controller is a means through which new content can be added to the edcontentmaster collection in MongoDB. Such content

include a mix of videos, quizzes, props, or other resources available for a particular stage or subject and their topics.

```
export const createMasteredcontent = (req, res) => {
  const {
    stage,
    subject,
    topic,
    topicid,
    contentset,
    quiz,
    subjectiamge,
    slcontent,
    sltopic,
    slsubject,
    topicimage,
    contentimage,
    isPremium,
    timeDuration,
    boardid,
    boardname,
    scholarshipid,
    scholarshipname,
    stageid,
    subjectid,
    videos,
    concepts,
    conceptpdfurl
  } = req.body;

  const contentid = Date.now();
  const content = new edcontentmaster({
    contentid,
    stage,
    subject,
    topic,
```

```
    contentset,
    slcontent,
    sltopic,
    slsubject,
    quiz,
    subjectiamge,
    topicimage,
    contentimage,
    isPremium,
    timeDuration,
    boardid,
    boardname,
    scholarshipid,
    scholarshipname,
    stageid,
    subjectid,
    topicid,
    videos,
    concepts,
    conceptpdfurl
  });
  edcontentmaster.findOne({ contentid: contentid }, (err, data) => {
    if (data) {
      return res.send({ message: "Content already available.", status: false });
    } else {
      content.save((err) => {
        if (err) {
          return res.send({ error: err, status: false });
        } else {
          return res.send({
            message: "Successfully added content.",
            status: true,
          });
```

How To Create API for getting Leader board:-

An API for a leaderboard would usually fetch a list of appended users who have scored or earned some points or percentage completion in a particular application. This can be a leaderboard on various parameters such as scores in quizzes, levels of progress achieved, or total scores in a particular learning application. Here is the process on how to develop such an API in detail going through each and every aspect of how the back end works, the database query structure and the design of the controller.

Controller for Leaderboard

The controller fetch the request to obtain the leaderboard data. It ought to interact with the leaderboard collection, rank the users according to their respective scores, and fetch the users with the highest ranks.

```
export const createmostprobableleaderboard = (req, res) => {
  const {
    childid,
    parentid,
    name,
    age,
    stage,
    stageid,
    mostprobablequestionid,
    subject,
    mostprobablequestionname,
    subjectid,
    ispremium,
    lastexamtotalmark,
    lastexamtotalsecurmark,
    lastexamtotalwrongmark,
    lastexamtotalskipmark,
    lastexampercentage,
    lastexamtimetaken,
    answerdetails,
    subspeanswerdetails,
  } = req.body;
  edmostprobableleaderboard.findOne(
    { childid: childid, mostprobablequestionid: mostprobablequestionid },
    (err, suc) => {
      if (suc) {
        res.send({
          message: "You already completed, please reattempt.",
          status: false,
        });
      } else {
        let numberofattempt = 1;
        let lastattemptDate = new Date();
        const childcontent = new edmostprobableleaderboard({
```

```
        const childcontent = new edmostprobableleaderboard({
          childid,
          parentid,
          name,
          age,
          stageid,
          stage,
          mostprobablequestionid,
          subject,
          mostprobablequestionname,
          subjectid,
          numberofattempt,
          ispremium,
          lastexamtotalmark,
          lastexamtotalsecurmark,
          lastexamtotalwrongmark,
          lastexamtotalskipmark,
          lastexamtimetaken,
          lastexampercentage,
          answerdetails,
          lastattemptDate,
          subspeanswerdetails,
        });
        childcontent.save((err, suc) => {
          if (err) {
            return res.send({ error: err, status: false });
          } else {
            return res.send({
              message: "Successfully added content.",
              status: true,
            });
          }
        });
      }
    }
  }
```

Conclusion:-

In this time, e-learning platforms are a critical component in enhancing the quality of education by giving learners a lot of information and materials easily. To create an efficient e-learning system, a back-end structure is very important since it is what provides reliability, scalability and security to a system for all users whether students, teachers or administration.

For example, the software architecture we chose to use for backend development includes Node.js, Express.js, and MongoDB with the addition of JWT for secure authorization in complex processes such as users login, content creation or storage, and data preservation. The confluence of models and schemas prevents the application data from becoming chaotic, thus allowing swift operation between different parts of the system.

The necessary functionality for learners that enables them to concentrate on the learning process without unnecessary distractions is also found So in order to help the users of the system, the back end does not stop with user registration and login (OTPs inclusive), student progress tracking is embedded in the system. Adding new content such as subjects or topics or quizzes enables our system to be dynamic thus catering for the needs of the users at all times.

Lastly, in relation to this, our platform has features such as a **leaderboard**, which promotes competition and the desire to accomplish objectives, hence sustaining the users' engagement and the drive to excel. In conclusion, it is worth mentioning that our backend system was constructed with more emphasis on scalability, security, and user experience; thus it supports the creation of an education application that provides quality and worth to its users.